sims

student information management system

Eeshah Rashid CT-213
Fatima Zaman CT-205
Hiba Abrar CT-204

CONTENTS

INTRODUCTION

Problem Statement

Managing student information manually is time-consuming, error-prone, and inefficient, especially in institutions handling large numbers of students. Tracking personal details, enrolled courses, attendance, and academic progress often requires multiple disconnected systems or manual paperwork, leading to data inconsistencies and difficulty in generating reports.

This project, the Student Information Management System (SIMS), addresses this problem by providing an integrated application with both a graphical desktop interface (built using Qt QML) and a robust backend (built in C++ with SQLite). Together, these components allow administrators and students to easily manage student records, course enrollments, attendance, and progress tracking in a centralized, user-friendly environment.

PROJECT SCOPE

The SIMS Project includes:

A graphical desktop application frontend developed in QML, offering an intuitive and interactive user interface

A backend system developed in C++ using SQLite for data storage, managing all core operations (CRUD for students, courses, attendance, progress)

Full integration between frontend and backend using CMake, ensuring seamless communication between the UI and the database layer

Local data handling, meaning all data is stored and accessed locally without requiring an internet connection

It does not cover:

Web or mobile platform support (the system runs as a desktop application only)

Cloud-based data storage or online synchronization

Advanced reporting or analytics beyond the available queries and displayed summaries

Integration with external services such as student portals, email systems, or third-party apps

PROJECT FEATURES

FUNCTIONAL REQUIREMENTS

- Student Management: Add, view, update, and delete student records (name, password, gender, email, phone, address).
- Course Management: Assign multiple courses to each student and retrieve the list of enrolled courses.
- Attendance Tracking: Record and view the number of classes a student has attended per course.
- Progress Tracking: Record and view the academic progress (percentage) for each student per course.

- Graphical User Interface (GUI): Provide a desktop interface built using QML for interacting with the system.
- Backend Integration: Connect frontend (QML) with backend (C++) through CMake, ensuring smooth data handling.
- Database Querying: Support command-line arguments like --dump to display all database contents for quick admin checks.

NON-FUNCTIONAL REQUIREMENTS

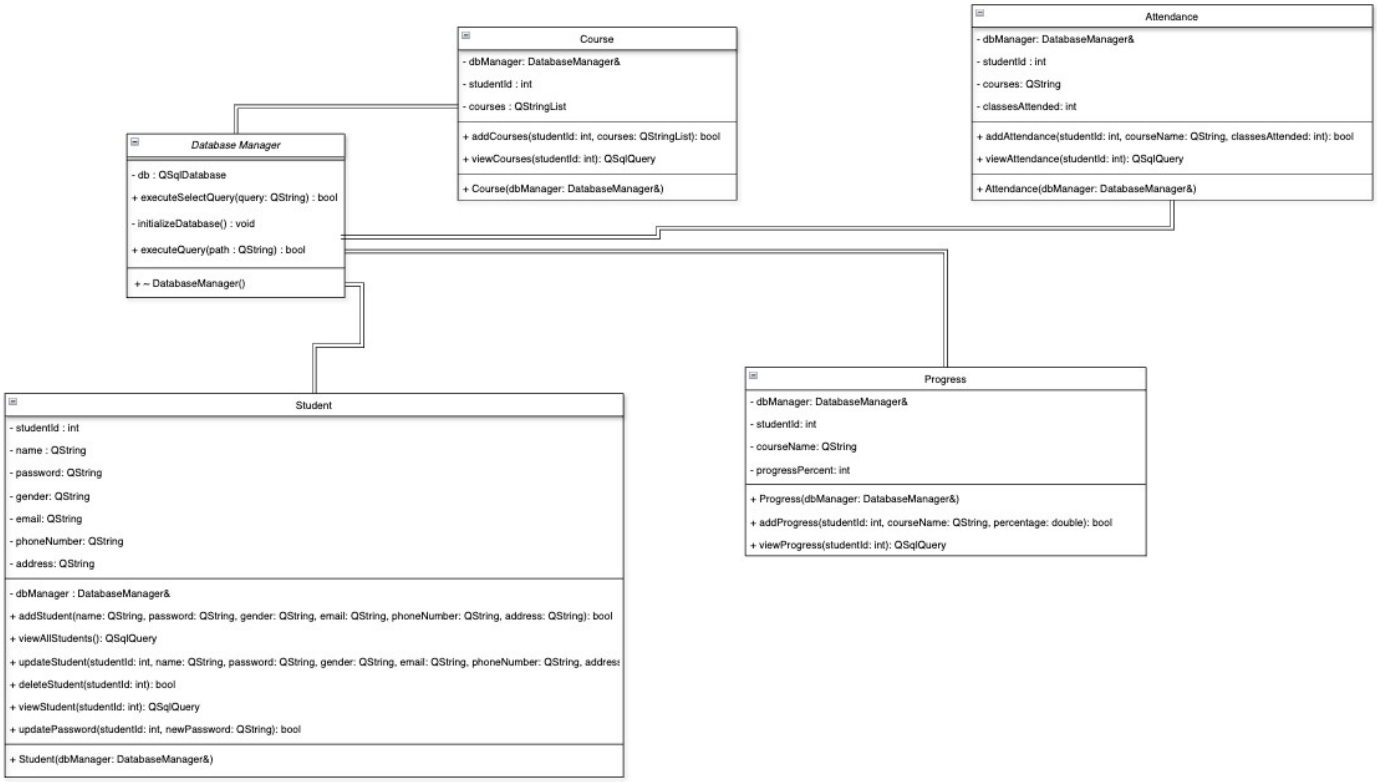
- Performance: Quick database access and responsive user interface for efficient user experience.
- Scalability: Suitable for small to medium datasets typical of educational institutions; not designed for very large-scale operations.
- Reliability: Ensures stable data storage and operations without crashes or unexpected behavior.
- Usability: User-friendly interface with intuitive navigation, reducing the learning curve.
- Portability: Compatible with major desktop platforms supported by Qt (Windows, Linux, macOS).
- Maintainability: Modular and well-structured code (separate classes and files) for easier updates and debugging.

TOOLS AND TECHNOLOGIES USED

- **Programming Language:** C++
- **Framework:** Qt (for both backend logic and frontend QML interface)
- **Database:** SQLite (managed through a custom DatabaseManager class)
- **Build System:** CMake (to integrate and compile backend and frontend components)
- **Development Environments:** Qt Creator, Visual Studio Code
- **Design Tool:** Qt Design Studio (for creating and refining QML-based UI)
- **Version Control:** Git (with GitHub for remote repository management)
- **Support Resources:** Stack Overflow (for troubleshooting and code guidance), Canva Pro (for preparing diagrams, visuals, and report materials)

IMPLEMENTATION

UML Diagram



DATABASE SCHEMA

The project uses a relational database with the following tables and relationships:

1. Students Table

Column Name	Data Type	Description
student_id	INTEGER	Primary Key, auto-incremented unique student identifier
name	TEXT	Name of the student
password	TEXT	Password for student account
gender	TEXT	Gender of the student
email	TEXT	Email address of the student
phone_number	TEXT	Contact number of the student
address	TEXT	Address of the student

Primary Key: student_id

2. Courses Table

Column Name	Data Type	Description
student_id	INTEGER	Foreign Key referencing Students.student_id
course_name	TEXT	Name of the course

Primary Key: (student_id, course_name)

Foreign Key: student_id → Students(student_id)

3. Attendance Table

Column Name	Data Type	Description
student_id	INTEGER	Foreign Key referencing Students.student_id
course_name	TEXT	Name of the course
classes_attended	INTEGER	Number of classes attended

Primary Key: (student_id, course_name)

Foreign Key: student_id → Students(student_id)

4. Progress Table

Column Name	Data Type	Description
student_id	INTEGER	Foreign Key referencing Students.student_id
course_name	TEXT	Name of the course
percentage	REAL	Progress percentage in the course

Primary Key: (student_id, course_name)

Foreign Key: student_id → Students(student_id)

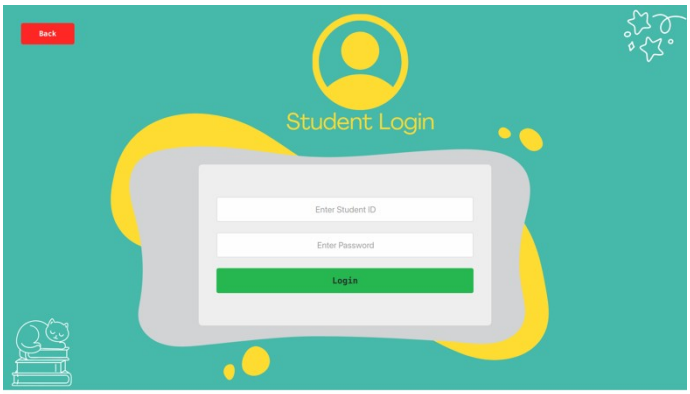
Relationships Between Tables

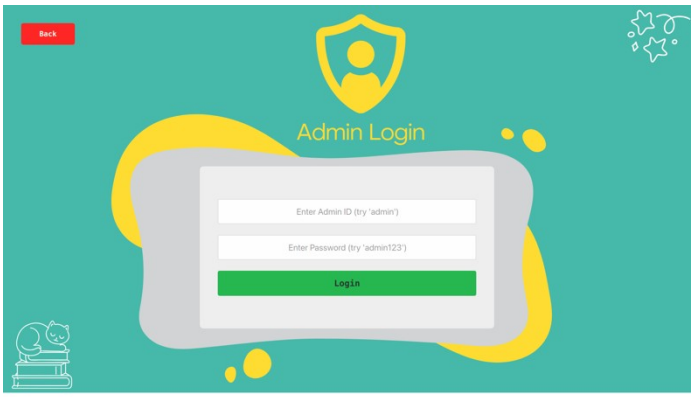
- Each student can have **multiple courses, attendance records, and progress records.**
- The student_id is used as a **foreign key** in Courses, Attendance, and Progress tables, linking them to the Students table.

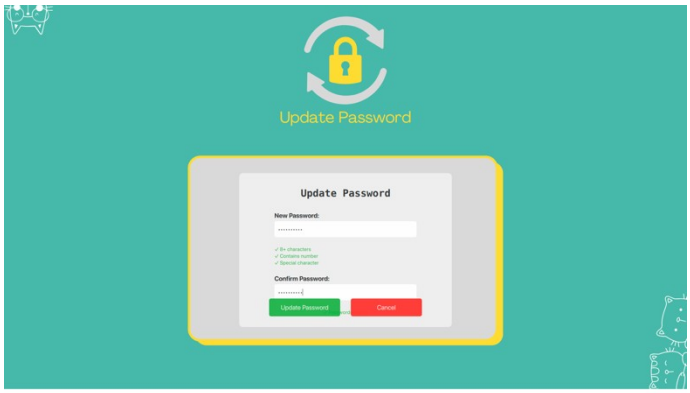
LIST OF CLASSES

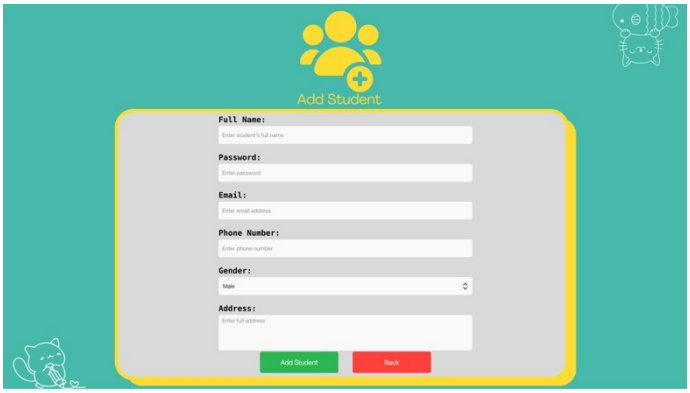
1. Student (student.cpp)

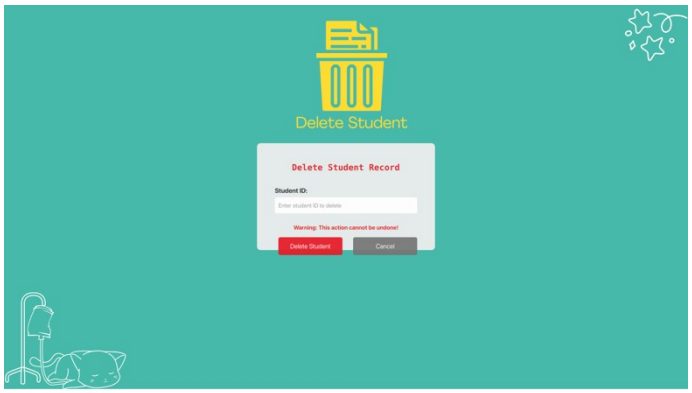
Handles all student-related operations like adding a new student, updating details, deleting a student, viewing student profiles, and updating passwords.

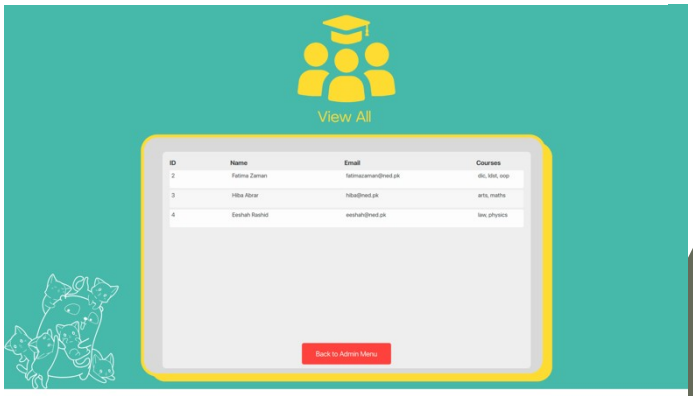


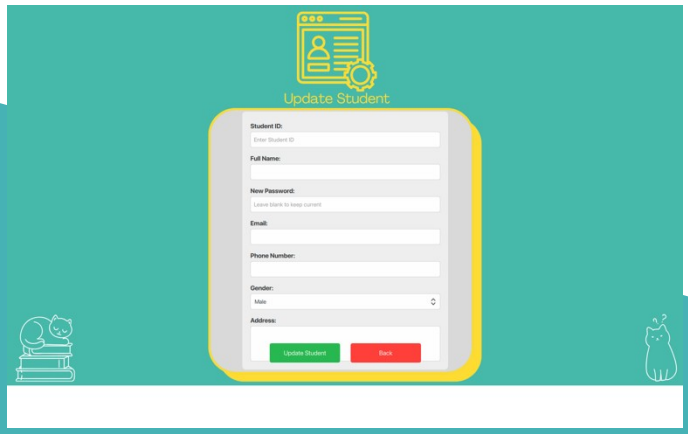


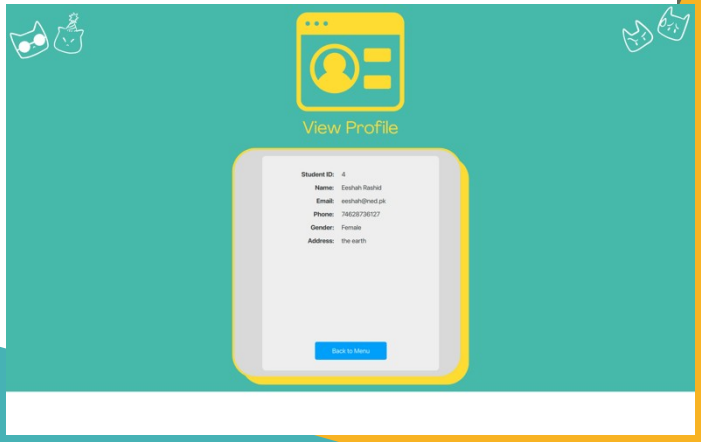








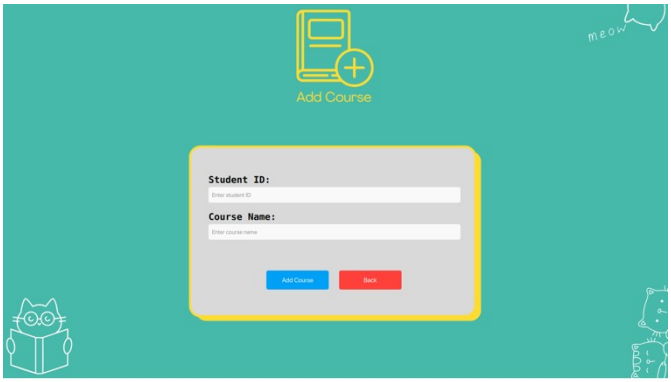


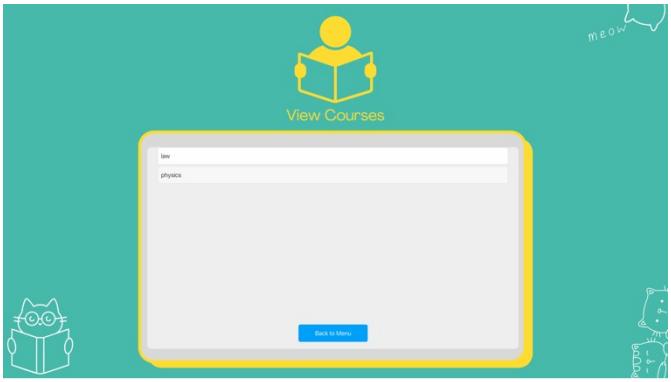


2. Course (course.cpp)

Manages the student's enrolled courses, including adding courses, checking if a course already exists, and retrieving the list of courses for a student.

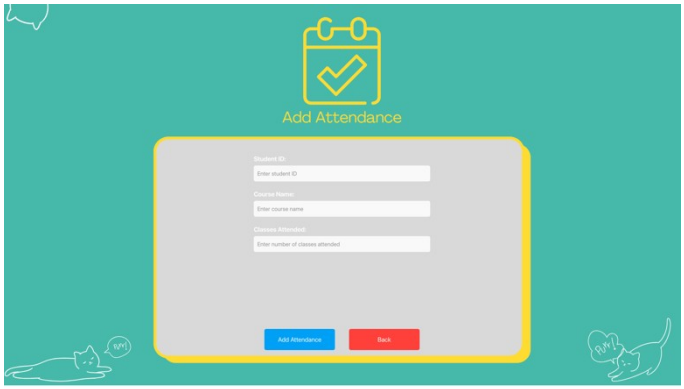
3. Attendance (attendance.cpp)

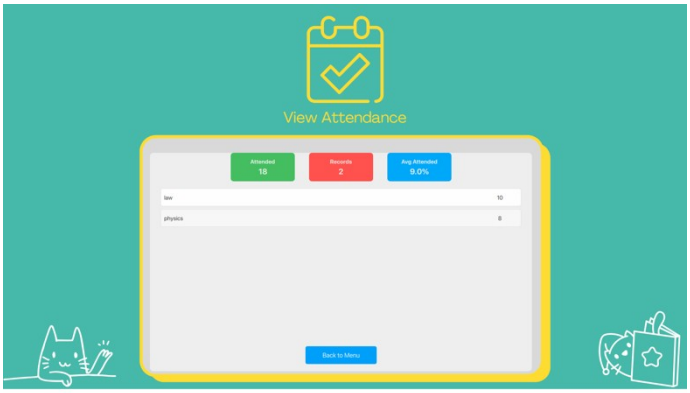




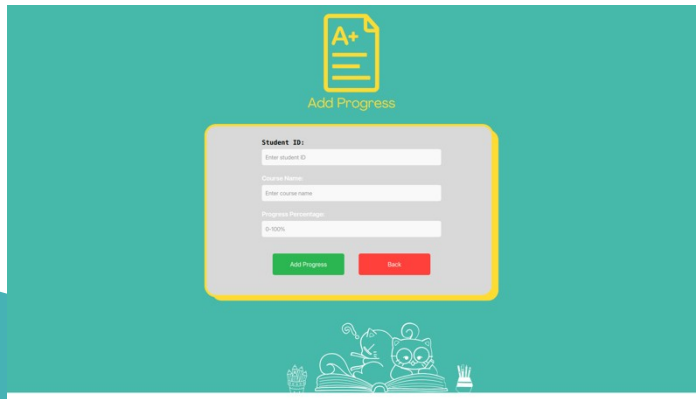
Tracks attendance records for students, including adding new attendance records (if they don't already exist) and viewing the number of classes attended per course.

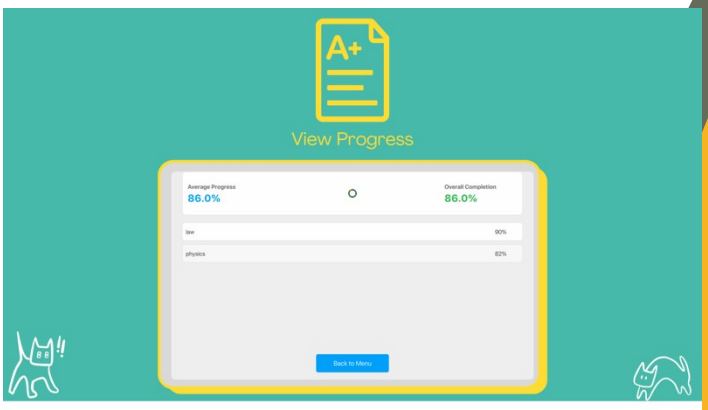
4. Progress (progress.cpp)





Manages the progress of students in their courses, including adding percentage-based progress data and retrieving progress reports per student.





5. DatabaseManager (databasemanager.cpp)

Handles all database operations, including connecting to the SQLite database, creating required tables, executing queries, and ensuring the database is initialized properly.

CODE:

Source files

attendance.cpp:

```
#include "attendance.h"


Attendance::Attendance(DatabaseManager& dbManager) : dbManager(dbManager) {}


// Add attendance for a student's course

bool Attendance::addAttendance(int studentId, const QString& courseName, int
classesAttended) {

    // Check if attendance already exists for the student and course

    QString checkQuery = QString("SELECT * FROM Attendance WHERE student_id = %1
AND course_name = '%2'")

                             .arg(QString::number(studentId), courseName);

    QSqlQuery checkResult = dbManager.executeSelectQuery(checkQuery);


    if (checkResult.next()) {

        qDebug() << "Attendance already exists for student ID" << studentId << "and
course" << courseName;

        return false; // Skip adding attendance

    }


    // Add attendance

    QString query = QString("INSERT INTO Attendance (student_id, course_name,
classes_attended) "

                            "VALUES (%1, '%2', %3)")

                        .arg(QString::number(studentId), courseName,
QString::number(classesAttended));

    return dbManager.executeQuery(query);
```

```
}


// View attendance for a student

QSqlQuery Attendance::viewAttendance(int studentId) {

    QString query = QString("SELECT course_name, classes_attended FROM Attendance
WHERE student_id = %1")

    .arg(QString::number(studentId));

    return dbManager.executeSelectQuery(query);

}
```

course.cpp:

```
#include "course.h"


Course::Course(DatabaseManager& dbManager) : dbManager(dbManager) {}


// Add courses for a student

bool Course::addCourses(int studentId, const QStringList& courses) {

    for (const QString& course : courses) {

        // Check if the course already exists for the student

        QString checkQuery = QString("SELECT * FROM Courses WHERE student_id = %1
AND course_name = '%2'")

                                    .arg(QString::number(studentId), course);

        QSqlQuery checkResult = dbManager.executeSelectQuery(checkQuery);


        if (checkResult.next()) {

            qDebug() << "Course" << course << "already exists for student ID" <<
studentId;

            continue; // Skip adding this course

        }


        // Add the course

        QString query = QString("INSERT INTO Courses (student_id, course_name) "

                                    "VALUES (%1, '%2')")
```

```
                            .arg(QString::number(studentId), course);
        if (!dbManager.executeQuery(query)) {
            return false;
        }
    }
    return true;
}
// View courses for a student
QSqlQuery Course::viewCourses(int studentId) {
    QString query = QString("SELECT course_name FROM Courses WHERE student_id =
%1")
    .arg(QString::number(studentId));
    return dbManager.executeSelectQuery(query);
}
```

databasemanager.cpp:

```
#include "databasemanager.h"


DatabaseManager::DatabaseManager(const QString& path) {
    db = QSqlDatabase::addDatabase("QSQLITE");
    db.setDatabaseName(path);


    if (!db.open()) {
        qDebug() << "Error: Could not open database.";
    } else {
        qDebug() << "Database connected successfully.";
        initializeDatabase();
    }
}


DatabaseManager::~DatabaseManager() {
    if (db.isOpen()) {
        db.close();
```

```
    }
}

bool DatabaseManager::executeQuery(const QString& query) {
    QSqlQuery q;
    if (!q.exec(query)) {
        qDebug() << "Query error:" << q.lastError();
        return false;
    }
    return true;
}

QSqlQuery DatabaseManager::executeSelectQuery(const QString& query) {
    QSqlQuery q;
    if (!q.exec(query)) {
        qDebug() << "Select query error:" << q.lastError();
    }
    return q;
}

void DatabaseManager::initializeDatabase() {
    // Create Students table
    executeQuery("CREATE TABLE IF NOT EXISTS Students ("
                 "student_id INTEGER PRIMARY KEY AUTOINCREMENT, "
                 "name TEXT NOT NULL, "
                 "password TEXT NOT NULL, "
                 "gender TEXT NOT NULL, "
                 "email TEXT NOT NULL, "
                 "phone_number TEXT NOT NULL, "
                 "address TEXT NOT NULL)");

    // Create Courses table
```

```
    executeQuery("CREATE TABLE IF NOT EXISTS Courses ("
                 "student_id INTEGER, "
                 "course_name TEXT NOT NULL, "
                 "FOREIGN KEY (student_id) REFERENCES Students(student_id), "
                 "PRIMARY KEY (student_id, course_name))");

    // Create Attendance table
    executeQuery("CREATE TABLE IF NOT EXISTS Attendance ("
                 "student_id INTEGER, "
                 "course_name TEXT NOT NULL, "
                 "classes_attended INTEGER, "
                 "FOREIGN KEY (student_id) REFERENCES Students(student_id), "
                 "PRIMARY KEY (student_id, course_name))");

    // Create Progress table
    executeQuery("CREATE TABLE IF NOT EXISTS Progress ("
                 "student_id INTEGER, "
                 "course_name TEXT NOT NULL, "
                 "percentage REAL, "
                 "FOREIGN KEY (student_id) REFERENCES Students(student_id), "
                 "PRIMARY KEY (student_id, course_name))");

    qDebug() << "Database initialized successfully.";
}
```

progress.cpp:

```
#include "progress.h"


Progress::Progress(DatabaseManager& dbManager) : dbManager(dbManager) {}


// Add progress for a student's course
bool Progress::addProgress(int studentId, const QString& courseName, double
percentage) {
```

```
    // Check if progress already exists for the student and course

    QString checkQuery = QString("SELECT * FROM Progress WHERE student_id = %1 AND
course_name = '%2'")

                              .arg(QString::number(studentId), courseName);

    QSqlQuery checkResult = dbManager.executeSelectQuery(checkQuery);



    if (checkResult.next()) {

        qDebug() << "Progress already exists for student ID" << studentId << "and
course" << courseName;

        return false; // Skip adding progress

    }



    // Add progress

    QString query = QString("INSERT INTO Progress (student_id, course_name,
percentage) "

                            "VALUES (%1, '%2', %3)")

                        .arg(QString::number(studentId), courseName,
QString::number(percentage));

    return dbManager.executeQuery(query);

}



// View progress for a student

QSqlQuery Progress::viewProgress(int studentId) {

    QString query = QString("SELECT course_name, percentage FROM Progress WHERE
student_id = %1")

    .arg(QString::number(studentId));

    return dbManager.executeSelectQuery(query);

}
```

student.cpp:

```
#include "student.h"



Student::Student(DatabaseManager& dbManager) : dbManager(dbManager) {}



// Add a new student
```

```
bool Student::addStudent(const QString& name, const QString& password, const
QString& gender,

                         const QString& email, const QString& phoneNumber, const
QString& address) {

    QString query = QString("INSERT INTO Students (name, password, gender, email,
phone_number, address) "

                            "VALUES ('%1', '%2', '%3', '%4', '%5', '%6')")

                        .arg(name, password, gender, email, phoneNumber, address);

    return dbManager.executeQuery(query);

}


// View all students

QSqlQuery Student::viewAllStudents() {

    QString query = "SELECT * FROM Students";

    return dbManager.executeSelectQuery(query);

}


// Update a student's details

bool Student::updateStudent(int studentId, const QString& name, const QString&
password, const QString& gender,

                            const QString& email, const QString& phoneNumber, const
QString& address) {

    QString query = QString("UPDATE Students SET name = '%1', password = '%2',
gender = '%3', "

                            "email = '%4', phone_number = '%5', address = '%6'
WHERE student_id = %7")

                        .arg(name, password, gender, email, phoneNumber, address,
QString::number(studentId));

    return dbManager.executeQuery(query);

}


// Delete a student

bool Student::deleteStudent(int studentId) {

    QString query = QString("DELETE FROM Students WHERE student_id = %1")

    .arg(QString::number(studentId));

    return dbManager.executeQuery(query);
```

```
}


// View a student's profile
QSqlQuery Student::viewStudent(int studentId) {
    QString query = QString("SELECT * FROM Students WHERE student_id = %1")
    .arg(QString::number(studentId));
    return dbManager.executeSelectQuery(query);
}


// Update a student's password
bool Student::updatePassword(int studentId, const QString& newPassword) {
    QString query = QString("UPDATE Students SET password = '%1' WHERE student_id =
%2")
    .arg(newPassword, QString::number(studentId));
    return dbManager.executeQuery(query);
}
```

Header files

attendance.h:

```
#ifndef ATTENDANCE_H
#define ATTENDANCE_H

#include <QString>
#include <QSqlQuery>
#include "databasemanager.h"

class Attendance {
public:
    Attendance(DatabaseManager& dbManager);

    // Admin Menu Methods
    bool addAttendance(int studentId, const QString& courseName, int
classesAttended);
```

```
    // Student Menu Methods
    QSqlQuery viewAttendance(int studentId);

private:
    DatabaseManager& dbManager;
};

#endif // ATTENDANCE_H
```

course.h:

```
#ifndef COURSE_H
#define COURSE_H

#include <QString>
#include <QStringList>
#include <QSqlQuery>
#include "databasemanager.h"

class Course {
public:
    Course(DatabaseManager& dbManager);

    // Admin Menu Methods
    bool addCourses(int studentId, const QStringList& courses);

    // Student Menu Methods
    QSqlQuery viewCourses(int studentId);

private:
    DatabaseManager& dbManager;
};

#endif // COURSE_H
```

progress.h:

```
#ifndef PROGRESS_H
#define PROGRESS_H

#include <QString>
#include <QSqlQuery>
#include "databasemanager.h"

class Progress {
public:
    Progress(DatabaseManager& dbManager);

    // Admin Menu Methods
    bool addProgress(int studentId, const QString& courseName, double percentage);

    // Student Menu Methods
    QSqlQuery viewProgress(int studentId);

private:
    DatabaseManager& dbManager;
};

#endif // PROGRESS_H
```

student.h:

```
#ifndef STUDENT_H
#define STUDENT_H

#include <QString>
#include <QSqlQuery>
#include "databasemanager.h"

class Student {
```

```
public:

    Student(DatabaseManager& dbManager);



    // Admin Menu Methods

    bool addStudent(const QString& name, const QString& password, const QString&
gender,

                    const QString& email, const QString& phoneNumber, const
QString& address);

    QSqlQuery viewAllStudents();

    bool updateStudent(int studentId, const QString& name, const QString& password,
const QString& gender,

                       const QString& email, const QString& phoneNumber, const
QString& address);

    bool deleteStudent(int studentId);



    // Student Menu Methods

    QSqlQuery viewStudent(int studentId);

    bool updatePassword(int studentId, const QString& newPassword);



private:

    DatabaseManager& dbManager;

};



#endif // STUDENT_H
```

databasemanager.h:

```
#ifndef DATABASEMANAGER_H

#define DATABASEMANAGER_H



#include <QSqlDatabase>

#include <QSqlQuery>

#include <QSqlError>

#include <QDebug>



class DatabaseManager {
```

```
public:
    DatabaseManager(const QString& path);
    ~DatabaseManager();

    bool executeQuery(const QString& query);
    QSqlQuery executeSelectQuery(const QString& query);

private:
    void initializeDatabase();
    QSqlDatabase db;
};

#endif // DATABASEMANAGER_H
```

FUTURE SCOPE

This project has great potential for further development and enhancement. Below are some key improvements and features that can be added in the future:

- Add a student login system with password recovery and profile management.
- Implement admin and teacher roles for managing courses, attendance, and progress.
- Integrate notifications (email/SMS) for attendance updates or progress alerts.
- Develop a mobile app version for students to check attendance and progress on the go.
- Improve the user interface with a modern, responsive design.
- Add data visualization (charts, graphs) for progress tracking.
- Include export options (PDF, Excel) for attendance and progress reports.

CONCLUSION AND LEARNING

- Through the development of this project, I gained hands-on experience in both frontend and backend development. On the frontend, I worked with **QML** and **Qt** to design an interactive user interface, improving my skills in UI/UX design and user interaction. On the backend, I implemented core functionalities using **C++** and managed the database with **SQLite**, learning how to design relational databases, write SQL queries, and ensure smooth integration between the frontend and backend. I also learned how to **build and launch a desktop application**, which gave me valuable insight into deploying software on real systems.

-

- Overall, this project strengthened my understanding of full-stack application development, enhanced my problem-solving skills, and gave me practical experience in developing and launching a complete, functional software system.

REFERENCES

Qt:

https://doc.qt.io/qt-6/qtquick-index.html

https://doc.qt.io/qtdesignstudio/index.html

https://doc.qt.io/qtcreator/index.html

https://doc.qt.io/qt-6/gettingstarted.html

https://www.youtube.com/watch?v=nep5QhQ1S14

https://www.youtube.com/watch?v=0aA7yzH0q0c

https://www.youtube.com/watch?v=UE0LJ7F3X1I

Cmake:

https://cmake.org/documentation/

https://www.geeksforgeeks.org/cmake-command/

https://github.com/ttroy50/cmake-examples

Others:

https://github.com/JesseTG/awesome-qt

https://stackoverflow.com/questions/tagged/qt

https://www.w3schools.com/cpp/